# DFLOW: ENABLING VERIFIER INFORMATION FLOW IN BLOCK DIFFUSION SPECULATIVE DECODING

**Yaojie Zhang**[1,3] **Linfeng Zhang**[4] **Bin Cui**[1,2] **Xupeng Miao**[1*]

[1]School of Computer Science & Beijing Key Laboratory of Software and Hardware Cooperative Artificial Intelligence Systems, Peking University
[2]Institute of Computational Social Science, Peking University (Qingdao)
[3]University of Electronic Science and Technology of China
[4]Shanghai Jiao Tong University

arXiv:2609.06498v1 [cs.CL] 6 Sep 2026

## ABSTRACT

Block diffusion speculative decoding improves LLM inference efficiency by proposing a block of future tokens in parallel and verifying them with a single forward pass through the target model. However, existing methods retain only the accepted prefix and discard the rejected suffix, preventing the computation spent on these positions from benefiting subsequent drafting rounds and forcing the drafter to repeatedly reconstruct representations for future tokens from scratch. We observe that rejection only determines whether a proposed token can be committed, while the verifier representations at rejected positions can still provide useful information for subsequent predictions. Based on this observation, we propose DFlow, a simple yet effective framework that enables verifier information to flow across drafting rounds. DFlow reuses the hidden states produced by the target verifier for the rejected suffix to guide subsequent drafting without additional target computation. To effectively learn this information flow across drafting rounds, we introduce a self-condition train strategy that feeds verifier representations from earlier predictions back into subsequent predictions. Experiments on Qwen3 models across diverse benchmarks demonstrate that DFlow consistently improves draft quality and acceptance length over DFlash.

## 1 INTRODUCTION

Large language models (LLMs) have achieved remarkable performance across a wide range of tasks (Singh et al., 2025; DeepSeek-AI, 2026; Kimi Team, 2026), but autoregressive decoding remains inherently sequential and often becomes a major bottleneck in LLM inference (Miao et al., 2025; Nie et al., 2025). Speculative decoding alleviates this bottleneck by using a lightweight drafter to propose multiple future tokens and verifying them in parallel with the target model (Leviathan et al., 2023; Li et al., 2026). Among recent approaches, block diffusion speculative decoding is particularly attractive because it predicts an entire block of future tokens simultaneously (Sandler et al., 2025; Liu et al., 2026a), substantially reducing the sequential overhead of drafting while preserving exact target model outputs through verification. Despite this efficiency, its practical speedup still depends heavily on how many proposed tokens can be accepted in each drafting round (Zhang et al., 2026a; Wu et al., 2026), making draft quality a key factor in decoding acceleration.

However, existing block diffusion speculative decoding methods impose an information discontinuity across consecutive drafting rounds. Given a verified draft block, only the accepted prefix is committed as context for the next drafting round, while the remaining suffix and its intermediate representations are discarded (Chen et al., 2026a; Huang et al., 2026). In the next drafting round, the drafter predicts another block of future positions, some of which overlap with positions already predicted and processed in the previous round. Nevertheless, these overlapping positions are reset to mask embeddings and reconstructed from scratch, preventing the intermediate information obtained earlier from being carried forward. As a result, the rejection boundary serves not only as a nec-

*Corresponding author.

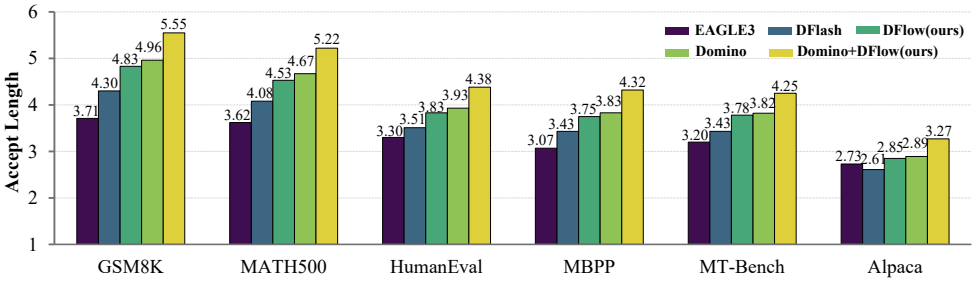



Figure 1: Average acceptance length of different methods on Qwen3-1.7B under greedy decoding.

essary commit boundary for lossless speculative decoding, but also as an unnecessary information boundary that blocks information flow across consecutive drafting rounds.

Recent work suggests that speculative execution provides value beyond the tokens that are eventually accepted. Drafting and verification naturally produce rich intermediate artifacts, including future token predictions, attention patterns, routing signals, and contextual representations, which have increasingly been reused to anticipate KV access, prefetch experts, or reduce subsequent computation (Liang et al., 2026; Chen et al., 2025; 2026b). These approaches reveal a broader principle: a speculative proposal is not merely a candidate output, but can also serve as a probe that exposes useful information throughout the inference process. This perspective naturally extends to target verification. Rejection only determines whether a proposed token can be committed; it does not imply that the verifier representation computed at that position is uninformative. A verifier hidden state is a continuous contextual representation produced by the target model and can retain useful information about the continuation even when the corresponding discrete proposal is rejected. This is particularly relevant to the rejected suffix, whose positions have already been processed by the target model before the verification outcome is finalized. Therefore, discarding these representations together with the rejected tokens may unnecessarily erase information that could benefit subsequent drafting. This motivates a simple question: can the verifier information already computed for rejected positions be carried forward to improve the next drafting round?

To this end, we propose DFlow, a framework that enables verifier information to flow across drafting rounds in block diffusion speculative decoding. Rather than predicting overlapping future positions again from scratch, DFlow carries forward the verifier hidden states associated with the rejected suffix and uses them to guide subsequent drafting. These representations are obtained from the existing verification process and therefore introduce no additional target model forward pass. To support this cross-round inference pattern, we further introduce Multi-Round Self-Conditioned Training, which explicitly unfolds consecutive drafting and verification rounds, allowing the training procedure to reproduce the verifier information flow that arises during DFlow inference. As shown in Figure 1, DFlow consistently improves draft quality over DFlash across diverse reasoning, coding, and dialogue tasks on Qwen3-1.7B, demonstrating the effectiveness of reusing verifier information beyond the rejection boundary.Our contributions are summarized as follows:

1. We identify an overlooked information discontinuity in block diffusion speculative decoding: while the rejection boundary determines token commitment, verifier representations beyond this boundary can still provide useful guidance for subsequent drafting.
2. We propose **DFlow**, a simple and effective framework that enables verifier information flow across drafting rounds by relaying rejected-suffix representations to corresponding positions in the next round, with Multi-Round Self-Conditioned Training to learn this information reuse.
3. We validate DFlow on Qwen3-1.7B, Qwen3-4B, and Qwen3-8B across diverse reasoning, coding, and dialogue benchmarks. Under greedy decoding, DFlow improves the average acceptance length over DFlash by **10.4%**, **13.2%**, and **13.4%** on the three model scales, respectively.

## 2 Related Work

**Speculative Decoding for Efficient LLM Inference.** Speculative decoding addresses the memory bound bottleneck of LLM decoding by amortizing target model memory accesses across multiple

candidate tokens through parallel verification in a single forward pass (Chen et al., 2023; Miao et al., 2023; Zhang et al., 2026b). A representative line of work is the EAGLE series (Li et al., 2024a;b; 2025b), which performs autoregressive speculative drafting conditioned on target model features. To reduce the sequential overhead of such autoregressive drafting, DFlash introduces block diffusion speculative decoding, generating an entire draft block in parallel with a single draft model forward pass (Chen et al., 2026a). As drafting latency decreases, draft quality becomes the primary bottleneck to further speedup, motivating recent efforts to strengthen block parallel drafters (Yang & Li, 2026). DFlare improves the use of target model representations by allowing each draft layer to learn its own combination of features from multiple target layers (Zhang et al., 2026a), which provides more suitable information for different draft depths and enables deeper draft models to scale effectively. Domino focuses on the weakened causal dependency modeling within a draft block and introduces a lightweight causal head to refine parallel predictions (Huang et al., 2026). DSpark and DFly further explore this direction and validate such dependency modeling mechanisms under practical serving workloads (Cheng et al., 2026; Liu et al., 2026b). More recently, DFlash 2 improves local dependency modeling with lightweight dynamic convolutions while preserving fully parallel drafting (Inco AI, 2026). Together, these methods improve draft quality and model capacity while maintaining the low drafting cost of block diffusion speculative decoding.

**Intermediate Information Reuse in Speculative Decoding.** Recent work has increasingly explored how intermediate information produced during speculative decoding can be reused beyond its immediate role in token generation or verification. On the draft side, speculative predictions, routing decisions, and hidden states have been exploited to anticipate KV access, prefetch experts, or support subsequent speculation (Liang et al., 2026; Li et al., 2025a; Chen et al., 2026b). On the verification side, attention patterns and KV importance exposed during target verification have been reused to guide sparse attention and context selection in later drafting steps (Zhao et al., 2026). However, existing reuse mechanisms mainly exploit intermediate artifacts to precompute or selectively perform subsequent operations, thereby reducing inference overhead. In contrast, DFlow reuses the contextual information induced by draft proposals during target verification to directly improve draft quality. Specifically, it propagates the corresponding verifier representations beyond the rejection boundary as guidance for the next drafting round, improving subsequent draft prediction.

## 3 PRELIMINARIES

To facilitate the understanding of DFlow, we first review the inference process of DFlash and introduce the notation used in this paper. DFlash performs generation through a sequence of speculative rounds, each consisting of block diffusion drafting and exact-match verification.

**Block Diffusion Drafting.** Let $\mathbf{y}_{1:t_r}$ denote the committed context at speculative round $r$, and let $K$ denote the number of future tokens drafted in parallel. DFlash conditions a lightweight block diffusion drafter on hidden representations extracted from the target model. Specifically, let $\mathcal{L} = \{\ell_1, \ldots, \ell_m\}$ denote a fixed set of target layers sampled from shallow to deep. The hidden states from these layers are concatenated and projected into a fused target context representation:

$$\mathbf{H}_T^{(r)} = \mathrm{RMSNorm}\left(\mathbf{W}_c \left[\mathbf{H}_T^{(r,\ell_1)}; \ldots; \mathbf{H}_T^{(r,\ell_m)}\right]\right), \tag{1}$$

where $\mathbf{H}_T^{(r,\ell)}$ denotes the hidden states extracted from target layer $\ell$, and $\mathbf{W}_c$ denotes the projection matrix. The fused representation $\mathbf{H}_T^{(r)}$ is then injected into the Key and Value projections of each draft layer, providing persistent conditioning from the committed context.

To predict the next $K$ tokens, DFlash initializes the future positions with mask embeddings and jointly predicts the draft block conditioned on the fused target context representation:

$$\hat{\mathbf{y}}^{(r)} = \mathcal{D}_\theta\left(\mathbf{E}_{\mathrm{mask}}^{(r)}; \mathbf{H}_T^{(r)}\right), \tag{2}$$

where $\mathcal{D}_\theta$ denotes the block diffusion drafter and $\mathbf{E}_{\mathrm{mask}}^{(r)}$ denotes the mask representations for the $K$ future positions. While $\mathbf{H}_T^{(r)}$ provides target information from the committed context, the future positions are initialized from mask embedding at each speculative round, even when some of them overlap with positions already drafted and processed in the preceding round.

**Exact Match Verification.** Given the draft block $\hat{\mathbf{y}}^{(r)}$, the target model verifies the proposed tokens from left to right using exact token matching. Let $a_r$ denote the number of consecutive proposals that match the corresponding target predictions before the first mismatch. Accordingly, the verified draft block can be partitioned as

$$\hat{\mathbf{y}}^{(r)} = \Big[ \underbrace{\hat{\mathbf{y}}^{(r)}_{1:a_r}}_{\text{accepted prefix}} , \underbrace{\hat{y}^{(r)}_{a_r+1}}_{\text{first rejection}} , \underbrace{\hat{\mathbf{y}}^{(r)}_{a_r+2:K}}_{\text{rejected suffix}} \Big]. \tag{3}$$

The first $a_r$ proposals are committed as the *accepted prefix*. At the first rejected position, the draft proposal is replaced by the corresponding target prediction, denoted by $b^{(r)}$ and referred to as the *bonus token*. The remaining proposals, referred to as the *rejected suffix*, are discarded after verification. The rejected suffix may overlap with the beginning of the subsequent drafting window, so some future positions are drafted from scratch again in the next round. Yet the information obtained at these positions is not retained across rounds, introducing an information discontinuity.

# 4 DFLOW

DFlow addresses the information discontinuity across speculative rounds by preserving verifier representations associated with rejected draft positions and propagating them into subsequent drafting. During inference, the verifier representations are propagated to their corresponding future positions and incorporated into the input of the subsequent drafting round. To accommodate this information flow across multiple drafting rounds, we further introduce a self-conditioning train strategy that trains the drafter under both standard mask prediction and prediction with verifier information.

## 4.1 INFORMATION DISCONTINUITY ACROSS DRAFTING ROUNDS

Existing block diffusion speculative decoding suffers from an information discontinuity across drafting rounds. During exact match verification, only the accepted prefix and the bonus token are committed, whereas the rejected suffix is discarded. However, positions in the rejected suffix remain unresolved after the current round and may need to be predicted again as generation proceeds. Although the target model has already processed these positions and produced contextual representations during verification, such information is discarded together with the rejected proposals. Consequently, the rejection boundary not only determines which tokens can be committed, but also becomes an information boundary between consecutive drafting rounds.

Importantly, rejection is a decision over discrete token commitment rather than a direct measure of information utility. Notably, tokens beyond the first rejection can still contain useful predictive signals, with some positions remaining consistent with the target continuation despite belonging to the rejected suffix. More importantly, during verification, each draft proposal participates in the target model's causal computation, inducing a contextual representation that integrates information from the preceding context and previously proposed tokens. Such representations are directly involved in predicting subsequent tokens and may therefore retain useful information even when the corresponding discrete proposals are rejected. This motivates the following hypothesis:

**Main Hypothesis.** Verifier representations at rejected positions can provide useful guidance for subsequent drafting, improving draft quality by preserving information that would otherwise be discarded.

## 4.2 VERIFIER INFORMATION RELAY

Based on the above hypothesis, DFlow relays verifier representations associated with the rejected suffix into subsequent drafting rounds and reuses them at the corresponding overlapping positions as auxiliary information, allowing these positions to inherit information from previous verification rather than reconstructing their representations from scratch in each round. To make the relayed information compatible with subsequent drafting, DFlow first transforms verifier states into the drafter hidden space while maintaining their original prediction correspondence, and then refines them according to the semantic change introduced at the rejection boundary.

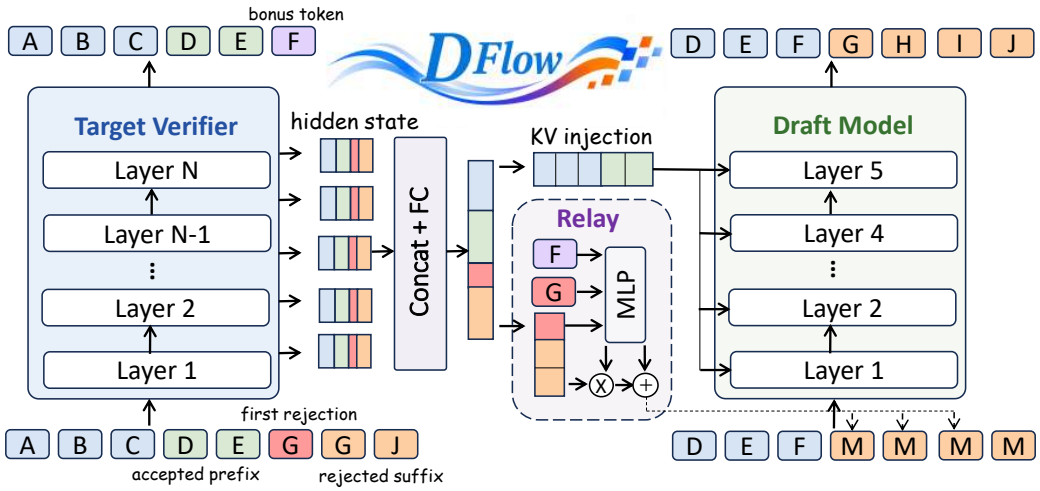



Figure 2: **Pipeline of DFlow.** DFlow reuses verifier hidden states from rejected suffix positions through the relay module to enhance the corresponding mask embeddings in the next drafting round, without an additional target model forward.

**Verifier Representation Alignment.** To make the verifier hidden states compatible with the drafter hidden space, DFlow shares the same feature projection used by DFlash for target context conditioning. Specifically, hidden states from the selected target layers over the rejected suffix are concatenated and transformed by the shared projection matrix $\mathbf{W}_c$ followed by RMSNorm, yielding verifier representations $\mathbf{h}_i^{(r)}$ for subsequent reuse. Since target verification already produces the required hidden states, DFlow requires no additional target model forward pass. Moreover, the reusable suffix states are jointly projected with the target features required by DFlash, introducing only a small number of additional positions.

The target verifier and the block diffusion drafter further differ in how their representations correspond to predicted tokens. The causal target model performs next token prediction, where the hidden state at position $i$ predicts the token at position $i+1$, whereas the drafter performs mask token prediction, where each masked position predicts the token at the same position. To align verifier representations with the drafter, DFlow shifts the verifier representations before injecting them into the drafter, so that both representations correspond to the same predicted token. The aligned representations are then injected into the corresponding unresolved positions of the subsequent draft.

**Reject Conditioned Draft Initialization.** Although the aligned verifier representations contain useful information about the rejected suffix, they are produced under a speculative context that differs from the context used in the subsequent drafting round. During verification, the rejected suffix is processed following the first rejected proposal $\hat{y}_{a_r+1}^{(r)}$, whereas the next round continues from the target correction $b^{(r)}$ at the same position. The difference between these two boundary tokens therefore provides a direct signal of how the context underlying the verifier representations changes across rounds.

DFlow uses this boundary information to determine how the verifier representation at each reusable position should be retained and corrected. Specifically, we combine the embeddings of the target correction and the first rejected proposal with $\mathbf{h}_i^{(r)}$, and use a lightweight MLP to produce channel-wise scaling and correction vectors:

$$\left[\boldsymbol{\beta}_i^{(r)}; \boldsymbol{\gamma}_i^{(r)}\right] = \mathrm{MLP}_\phi\left(\left[\mathbf{E}\left(b^{(r)}\right); \mathbf{E}\left(\hat{y}_{a_r+1}^{(r)}\right); \mathbf{h}_i^{(r)}\right]\right), \tag{4}$$

where $\boldsymbol{\beta}_i^{(r)}$ controls the contribution of the verifier representation and $\boldsymbol{\gamma}_i^{(r)}$ provides an additive correction. This channel-wise modulation allows DFlow to selectively preserve, suppress, or adjust the relayed information according to the change at the rejection boundary.

The verifier representation is first normalized and then adjusted by the learned scaling and correction vectors before being added to the standard mask embedding, forming the input for the subsequent

drafting round:

$$\mathbf{x}_i^{(r+1)} = \mathbf{E}_{\text{mask}} + \boldsymbol{\beta}_i^{(r)} \odot \text{LN}\left(\mathbf{h}_i^{(r)}\right) + \boldsymbol{\gamma}_i^{(r)}, \tag{5}$$

where $\text{LN}(\cdot)$ denotes LayerNorm. This augmented input is used only at positions with valid reusable verifier representations, while the remaining positions continue to use the standard mask embedding. In the first drafting round, where no verifier information is available, or when the previous verification produces no reusable rejected suffix, DFlow simply uses mask embeddings as the draft inputs.

### 4.3 Multi-Round Self-Conditioned Training

Standard DFlash training treats each sampled drafting round independently and initializes every new round from mask embedding, whereas DFlow inference conditions later drafts on verifier representations produced in preceding rounds. To bridge this gap, DFlow introduces Multi-Round Self-Conditioned Training, consisting of a Multi-Round Rollout that explicitly unfolds consecutive drafting and verification rounds to propagate verifier information, and a Trajectory-Guided Transition that determines the starting position of each subsequent round by matching draft proposals with the training trajectory.

**Multi-Round Rollout.** Following DFlash, we sample anchor positions from trajectories generated by the target model and construct multiple draft blocks that can be packed for parallel training. For each sampled anchor, DFlow extends the original single round training into a sequence of consecutive drafting rounds. The initial round follows standard DFlash training and predicts the draft block from mask embeddings, preserving the fundamental mask prediction capability of the drafter while producing the proposals required for subsequent verification. These proposals are processed by the frozen target model to obtain verifier representations over the rejected suffix, which are then relayed to the next round. Subsequent rounds perform reject conditioned prediction, allowing the drafter to learn how to exploit verifier information from the preceding verification rather than relying solely on mask embeddings. The reject conditioned stage can continue for multiple rounds, with the rollout depth $k$ controlling how many such rounds are included during training. All unfolded rounds share the same drafter parameters, and their prediction losses jointly optimize this shared parameter set. In practice, we unfold $k = 2$ reject conditioned rounds after the initial mask prediction.

**Target Trajectory Transition.** Each transition in the multi-round rollout requires determining both where the next drafting round begins and what verifier information is carried forward from the current round. The starting position of the next round depends on the acceptance result of the current draft, with the new round anchor placed at the bonus token corresponding to the first mismatch. Although target verification can directly determine the accepted prefix and bonus token, its verification result during training may occasionally differ from the precomputed target trajectory due to sampling or numerical differences. We therefore determine the round transition by exact matching the draft proposals against the target trajectory and use the target token at the first mismatch as the bonus token. This keeps subsequent rounds aligned with the same target trajectory, allowing the target context computed for each training sample to be directly reused while verification in later rounds only processes the newly proposed block. Importantly, the target trajectory is used only to determine the transition between consecutive training rounds. The verifier information carried into the next round remains tied to the actual draft proposals produced in the current round. These proposals are processed by the frozen target model, and the verifier representations associated with the rejected suffix are extracted from this verification pass for subsequent reject conditioned prediction.

Following DFlash, we apply an exponential decay to the cross entropy loss as the draft position moves farther from the anchor, assigning greater importance to earlier predictions within each block. The overall training objective sums the loss of the initial mask prediction and those of all subsequent reject conditioned rounds:

$$\mathcal{L}_{\text{DFlow}} = \mathcal{L}^{(0)} + \sum_{q=1}^{k} \mathcal{L}^{(q)}, \tag{6}$$

where $k$ denotes the rollout depth, $\mathcal{L}^{(0)}$ is the loss of the initial mask prediction, and $\mathcal{L}^{(q)}$ for $q \geq 1$ denotes the loss of the $q$-th reject conditioned round. The losses from all rounds are summed before backpropagation and jointly optimize the same shared trainable drafter parameters. The target model remains frozen throughout training, while the verifier representations and round transitions

Table 1: Average acceptance length $\tau$ under the low-concurrency setting. Results are reported under both greedy decoding ($T = 0$) and sampling decoding ($T = 1$). The average is computed over all evaluated benchmarks.

| | | Math | | Code | | Chat | | Overall |
|---|---|---|---|---|---|---|---|---|
| **Model** | **Method** | **GSM8K** | **MATH-500** | **HumanEval** | **MBPP** | **MT-Bench** | **Alpaca** | **Avg.** |
| *Temperature = 0* | | | | | | | | |
| Qwen3-1.7B | EAGLE-3 | 3.71 | 3.62 | 3.30 | 3.07 | 3.20 | 2.73 | 3.27 |
| | DFlash | 4.30 | 4.08 | 3.51 | 3.43 | 3.43 | 2.61 | 3.56 |
| | DFlow | **4.83** | **4.53** | **3.83** | **3.75** | **3.78** | **2.85** | **3.93** |
| Qwen3-4B | EAGLE-3 | 3.30 | 3.14 | 3.08 | 3.00 | 3.07 | 2.86 | 3.08 |
| | DFlash | 4.41 | 4.39 | 3.86 | 3.61 | 3.39 | 2.61 | 3.71 |
| | DFlow | **5.13** | **4.92** | **4.33** | **4.12** | **3.80** | **2.90** | **4.20** |
| Qwen3-8B | EAGLE-3 | 3.25 | 3.12 | 3.25 | 2.86 | 2.88 | 2.61 | 3.00 |
| | DFlash | 4.40 | 4.36 | 3.82 | 3.55 | 3.21 | 2.61 | 3.66 |
| | DFlow | **5.10** | **4.94** | **4.28** | **4.08** | **3.58** | **2.93** | **4.15** |
| *Temperature = 1* | | | | | | | | |
| Qwen3-1.7B | EAGLE-3 | 3.55 | 3.45 | 3.20 | 3.08 | 3.03 | **2.62** | 3.16 |
| | DFlash | 3.66 | 3.34 | 2.96 | 2.87 | 2.82 | 2.27 | 2.99 |
| | DFlow | **4.04** | **3.65** | **3.21** | **3.17** | **3.04** | 2.42 | **3.26** |
| Qwen3-4B | EAGLE-3 | 3.24 | 3.01 | 3.02 | 2.95 | 2.97 | **2.77** | 2.99 |
| | DFlash | 3.84 | 3.53 | 3.33 | 3.12 | 2.77 | 2.32 | 3.15 |
| | DFlow | **4.40** | **3.97** | **3.70** | **3.55** | **3.13** | 2.49 | **3.54** |
| Qwen3-8B | EAGLE-3 | 3.15 | 2.94 | 3.07 | 2.80 | 2.73 | 2.48 | 2.86 |
| | DFlash | 3.84 | 3.54 | 3.21 | 3.06 | 2.67 | 2.30 | 3.10 |
| | DFlow | **4.31** | **3.96** | **3.58** | **3.44** | **2.92** | **2.50** | **3.45** |

are detached from the computation graph, preventing gradients from propagating through target verification or across transitions between consecutive rounds.

# 5 Experiments

## 5.1 Experimental Setup

**Models and Evaluation.** We evaluate DFlow on Qwen3-1.7B, Qwen3-4B, and Qwen3-8B across three representative task categories: mathematical reasoning, code generation, and open-ended dialogue. For mathematical reasoning, we use GSM8K (Cobbe et al., 2021), MATH-500 (Hendrycks et al., 2021); for code generation, we evaluate on HumanEval (Chen et al., 2021), MBPP (Austin et al., 2021); and for dialogue, we use MT-Bench (Zheng et al., 2023) and Alpaca (Taori et al., 2023).We compare DFlow with vanilla autoregressive decoding and representative speculative decoding methods, including EAGLE-3 (Li et al., 2025b) and DFlash (Chen et al., 2026a).

**Training and Implementation Details.** We train a DFlash drafter with five layers and a draft block size of 16 from scratch for all target models. We use 80K samples from ShareGPT[1] for training. Unless otherwise specified, Multi-Round Self-Conditioned Training uses $k = 2$ reject conditioned rounds following the initial mask prediction. All experiments are conducted on a single node with eight NVIDIA A800 GPUs.

## 5.2 Main Results

**Low Concurrency.** Table 1 reports the average acceptance length $\tau$ across all evaluated benchmarks under both greedy and sampling decoding. Across different model scales and task categories,

[1] https://huggingface.co/datasets/anon8231489123/ShareGPT_Vicuna_unfiltered

Table 2: High-concurrency throughput on Qwen3 models. Baseline rows report absolute throughput in TPS. Other entries report TPS with green subscripts indicating speedup over the corresponding baseline.

| Task | Method | Concurrency | | | | | |
|---|---|---|---|---|---|---|---|
| | | 1 | 2 | 4 | 8 | 16 | 32 |
| **Qwen3-4B** | | | | | | | |
| GSM8K | Baseline | 146.25 | 287.58 | 549.67 | 1063.16 | 1804.22 | 2850.74 |
| | EAGLE-3 | 233.99$_{1.60\times}$ | 420.16$_{1.46\times}$ | 758.99$_{1.38\times}$ | 1210.98$_{1.14\times}$ | 1634.96$_{0.91\times}$ | 1942.57$_{0.68\times}$ |
| | DFlash | 349.59$_{2.39\times}$ | 635.08$_{2.21\times}$ | 1124.58$_{2.05\times}$ | 1802.80$_{1.70\times}$ | 2368.26$_{1.31\times}$ | 2672.69$_{0.94\times}$ |
| | DFlow | **376.26**$_{2.57\times}$ | **679.39**$_{2.36\times}$ | **1217.36**$_{2.21\times}$ | **1959.07**$_{1.84\times}$ | **2580.66**$_{1.43\times}$ | **3208.95**$_{1.13\times}$ |
| HumanEval | Baseline | 145.58 | 287.16 | 544.38 | 1015.14 | 1742.29 | 2427.99 |
| | EAGLE-3 | 220.83$_{1.52\times}$ | 398.01$_{1.39\times}$ | 742.55$_{1.36\times}$ | 1201.58$_{1.18\times}$ | 1599.72$_{0.92\times}$ | 1888.08$_{0.78\times}$ |
| | DFlash | 334.06$_{2.29\times}$ | 604.59$_{2.11\times}$ | 1066.61$_{1.96\times}$ | 1750.33$_{1.72\times}$ | 2336.69$_{1.34\times}$ | 2835.77$_{1.17\times}$ |
| | DFlow | **343.42**$_{2.36\times}$ | **617.27**$_{2.15\times}$ | **1107.12**$_{2.03\times}$ | **1850.42**$_{1.82\times}$ | **2482.46**$_{1.42\times}$ | **3089.40**$_{1.27\times}$ |
| **Qwen3-8B** | | | | | | | |
| GSM8K | Baseline | 91.91 | 181.31 | 356.37 | 662.73 | 1070.40 | 1630.10 |
| | EAGLE-3 | 170.03$_{1.85\times}$ | 310.70$_{1.71\times}$ | 545.32$_{1.53\times}$ | 930.10$_{1.40\times}$ | 1244.19$_{1.16\times}$ | 1396.23$_{0.86\times}$ |
| | DFlash | 260.26$_{2.83\times}$ | 473.12$_{2.61\times}$ | 877.87$_{2.46\times}$ | 1404.98$_{2.12\times}$ | 1835.42$_{1.71\times}$ | 1891.10$_{1.16\times}$ |
| | DFlow | **283.47**$_{3.08\times}$ | **508.97**$_{2.81\times}$ | **944.11**$_{2.65\times}$ | **1518.16**$_{2.29\times}$ | **2023.22**$_{1.89\times}$ | **2198.17**$_{1.35\times}$ |
| HumanEval | Baseline | 91.86 | 181.98 | 352.96 | 663.87 | 1200.25 | 2036.43 |
| | EAGLE-3 | 168.83$_{1.84\times}$ | 306.27$_{1.68\times}$ | 565.86$_{1.60\times}$ | 947.66$_{1.43\times}$ | 1296.97$_{1.08\times}$ | 1467.19$_{0.72\times}$ |
| | DFlash | 230.75$_{2.51\times}$ | 422.87$_{2.32\times}$ | 809.74$_{2.29\times}$ | 1296.37$_{1.95\times}$ | 1777.93$_{1.48\times}$ | 1986.28$_{0.98\times}$ |
| | DFlow | **245.12**$_{2.67\times}$ | **448.55**$_{2.46\times}$ | **849.26**$_{2.41\times}$ | **1368.45**$_{2.06\times}$ | **1864.77**$_{1.55\times}$ | **2101.42**$_{1.03\times}$ |

Table 3: Ablation of verifier information relay.

| Training | Mask | Relay |
|---|---|---|
| DFlash | 3.56 | – |
| DFlash + Self-Conditioning | 3.67 | – |
| **DFlow** | 3.66 | **3.93** |

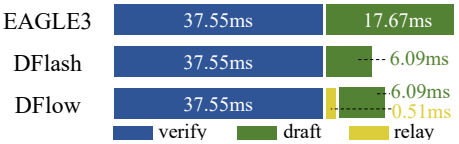



Figure 3: Runtime overhead.

DFlow consistently improves upon DFlash, demonstrating that verifier information from rejected suffixes provides useful guidance for subsequent drafting. Under greedy decoding ($T = 0$), DFlow increases the average acceptance length from **3.56** to **3.93**, **3.71** to **4.20**, and **3.66** to **4.15** on Qwen3-1.7B, Qwen3-4B, and Qwen3-8B, respectively. Similar improvements are observed under sampling decoding ($T = 1$), where the average acceptance length increases from **2.99** to **3.26**, **3.15** to **3.54**, and **3.10** to **3.45**. These consistent gains across reasoning, coding, and dialogue benchmarks show that DFlow effectively improves draft quality across different model scales and decoding settings.

**High Concurrency.** We further evaluate DFlow under concurrent serving workloads using SGLang. Table 2 reports serving throughput on Qwen3-4B and Qwen3-8B under concurrency levels from 1 to 32. Across both model scales and workloads, DFlow consistently achieves higher throughput than DFlash, demonstrating that improvements in draft quality translate into practical serving efficiency. The throughput gains persist across the entire concurrency range and, in several cases, become larger under heavier serving loads. This suggests that the additional computation introduced by verifier information relay is small relative to the benefit of accepting more draft tokens, allowing DFlow to remain effective as batching more fully utilizes the target model.

### 5.3 Ablation Studies

**Effect of Verifier Information Relay.** To quantify the respective contributions of Multi-Round Self-Conditioned Training and verifier information relay, we conduct controlled ablations. We first retain DFlow's multi-round rollout while resetting each new round to standard mask embeddings, thereby isolating the benefit of multi-round mask prediction from verifier information reuse. This improves the average acceptance length from 3.56 to 3.67, indicating that multi-round training alone provides only a modest gain. We then evaluate the trained DFlow model with verifier relay disabled, where all draft positions are initialized from the standard mask embedding. Its acceptance length of 3.66

closely matches the 3.67 achieved by multi-round mask training, showing that DFlow builds upon a strong mask token prediction capability. Finally, enabling verifier information relay increases the acceptance length from 3.66 to 3.93 under the same trained model. This clear improvement confirms that verifier information reuse is the primary source of DFlow's gain.

**Runtime Overhead Analysis.** We decompose the latency of each speculative round to quantify the additional cost introduced by DFlow. As shown in Figure 3, the dominant computation remains draft model execution and target verification, while verifier information relay introduces only 0.51 ms of additional latency, corresponding to merely **1.2%** overhead over DFlash. This low overhead is because the required hidden states are already produced during target verification and DFlow reuses the target feature projection of DFlash; the additional computation mainly consists of processing a small number of reusable rejected positions and applying the lightweight relay module. Compared with the substantial improvement in acceptance length, this additional cost is negligible, allowing the improved draft quality to translate into higher end-to-end decoding throughput.

## 6 Compatibility with Domino

Recent studies have shown that autoregressive heads can effectively recover causal dependencies among parallel draft tokens and substantially improve draft quality (Cheng et al., 2026). We therefore examine whether DFlow remains beneficial when combined with this stronger drafting paradigm, using Domino as a representative method that decouples causal dependency modeling from expensive autoregressive draft execution. Domino progressively refines predictions across positions within each drafting round (Huang et al., 2026), whereas DFlow propagates verifier information across rounds. To combine the two, we introduce an *online rollout* during DFlow training to ensure that the rejected suffix used for relay construction follows the same draft trajectory as inference.

Table 4: Compatibility with Domino.

| **Method** | $\tau$ |
|---|---|
| DFlash | 3.56 |
| Domino | 4.02 |
| + DFlow (w/o Online Rollout) | 2.22 |
| **+ DFlow (Online Rollout)** | **4.50** |

Domino is trained with teacher forcing, where each causal prediction is conditioned on the ground truth prefix and contributes to the cross entropy objective of the current round. Directly using these predictions to determine the rejected suffix and extract its verifier representations for DFlow, however, introduces a substantial mismatch with inference, where each position is conditioned on previously generated draft tokens. Teacher forcing maintains considerably higher accuracy at later draft positions, making the resulting rejected suffix overly optimistic for relay training. We therefore retain teacher forcing for optimizing Domino, while separately rolling out its actual predictions to determine the rejection boundary and extract the corresponding verifier representations for DFlow. Without online rollout, the average acceptance length drops to $2.22$. With online rollout, DFlow and Domino can be effectively combined, as shown in Table 4. On Qwen3 1.7B at $T = 0$, Domino achieves an average acceptance length of $4.02$, compared with $3.56$ for DFlash, while adding DFlow further improves it to $4.50$. This result shows that verifier information reuse remains effective when combined with stronger causal dependency modeling.

## 7 Conclusion

We propose DFlow, a block diffusion speculative decoding framework that enables verifier information to flow across consecutive drafting rounds. DFlow relays verifier representations from the rejected suffix to subsequent unresolved positions and introduces Multi-Round Self-Conditioned Training to learn this information reuse, without requiring additional target model forward passes. Experiments on Qwen3 models across diverse benchmarks show consistent gains over DFlash in acceptance length and decoding efficiency, while remaining compatible with stronger drafting methods such as Domino. These results suggest that verifier information beyond the rejection boundary can be effectively reused to improve subsequent drafting for LLM inference acceleration.

## REFERENCES

Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.

Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. Accelerating large language model decoding with speculative sampling. *arXiv preprint arXiv:2302.01318*, 2023.

Jian Chen, Yesheng Liang, and Zhijian Liu. Dflash: Block diffusion for flash speculative decoding. *arXiv preprint arXiv:2602.06036*, 2026a.

Liangkun Chen, Zijian Wen, Tian Wu, Xiaoxi Zhang, and Chuan Wu. Sp-moe: Speculative decoding and prefetching for accelerating moe-based model inference. *arXiv preprint arXiv:2510.10302*, 2025.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

Yuetao Chen, Xuliang Wang, Xinzhou Zheng, Ming Li, Peng Wang, and Hong Xu. Make every draft count: Hidden state based speculative decoding. *arXiv preprint arXiv:2602.21224*, 2026b.

Xin Cheng, Xingkai Yu, Chenze Shao, Jiashi Li, Yunfan Xiong, Yi Qian, Jiaqi Zhu, Shirong Ma, Xiaokang Zhang, Jiasheng Ye, et al. Dspark: Confidence-scheduled speculative decoding with semi-autoregressive generation. *arXiv preprint arXiv:2607.05147*, 2026.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

DeepSeek-AI. Deepseek-v4: Towards highly efficient million-token context intelligence. *CoRR*, abs/2606.19348, 2026. doi: 10.48550/ARXIV.2606.19348. URL https://doi.org/10.48550/arXiv.2606.19348.

Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the math dataset. *arXiv preprint arXiv:2103.03874*, 2021.

Jianuo Huang, Yaojie Zhang, Qituan Zhang, Hao Lin, Hanlin Xu, and Linfeng Zhang. Domino: Decoupling causal modeling from autoregressive drafting in speculative decoding. *arXiv preprint arXiv:2605.29707*, 2026.

Inco AI. Dflash 2: Keep drafting parallel, August 2026. URL https://inco.ai/blog/dflash2/.

Kimi Team. Kimi K3: open frontier intelligence. *CoRR*, abs/2607.24653, 2026. doi: 10.48550/ARXIV.2607.24653. URL https://doi.org/10.48550/arXiv.2607.24653.

Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pp. 19274–19286. PMLR, 2023.

Yan Li, Pengfei Zheng, Shuang Chen, Zewei Xu, Yuanhao Lai, Yunfei Du, and Zhengang Wang. Speculative moe: Communication efficient parallel moe inference with speculative token and expert pre-scheduling. *arXiv preprint arXiv:2503.04398*, 2025a.

Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. EAGLE: Speculative sampling requires rethinking feature uncertainty. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pp. 28935–28948. PMLR, 2024a.

Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. EAGLE-2: Faster inference of language models with dynamic draft trees. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pp. 7421–7432. Association for Computational Linguistics, 2024b. doi: 10.18653/v1/2024.emnlp-main.422.

Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. EAGLE-3: Scaling up inference acceleration of large language models via training-time test. In *Advances in Neural Information Processing Systems*, 2025b.

Zikun Li, Zhuofu Chen, Remi Delacourt, Gabriele Oliaro, Zeyu Wang, Qinghan Chen, Shuhuai Lin, April Yang, Zhihao Zhang, Zhuoming Chen, Yi-Hsiang Lai, Xinhao Cheng, Xupeng Miao, and Zhihao Jia. Adaserve: Accelerating multi-slo LLM serving with slo-customized speculative decoding. In Antonio Barbalace, Luo Mai, Roxana Geambasu, and Peter R. Pietzuch (eds.), *Proceedings of the 21st European Conference on Computer Systems, EuroSys 2026, McEwan Hall/The University of Edinburgh, Edinburgh, Scotland, UK, April 27-30, 2026*, pp. 36–54. ACM, 2026. doi: 10.1145/3767295.3769315. URL https://doi.org/10.1145/3767295.3769315.

Zuning Liang, Zhiyi Yao, Qi Chen, Yuedong Xu, Hao Dai, Zhiqiang Ding, Tongkai Yang, Jinlong Hou, and Yuan Cheng. Dualdecoder: Accelerate long context llm inference by predictive prefetch. *arXiv preprint arXiv:2607.26475*, 2026.

Fuliang Liu, Xue Li, Ketai Zhao, Yinxi Gao, Ziyan Zhou, Zhonghui Zhang, Zhibin Wang, Wanchun Dou, Sheng Zhong, and Chen Tian. Dart: Diffusion-inspired speculative decoding for fast llm inference. *arXiv preprint arXiv:2601.19278*, 2026a.

Hong Liu, Rui Cen, Junhan Shi, Guangshuo Qin, Jiebin Zhang, Tianyu Liu, Runzhi Fan, Guoliang Zhao, Ruobing Xie, Kai Zhang, et al. Angelspec: Towards real-world high performance inference with speculative decoding. *arXiv preprint arXiv:2607.25852*, 2026b.

Xupeng Miao, Gabriele Oliaro, Zhihao Zhang, Xinhao Cheng, Zeyu Wang, Zhengxin Zhang, Rae Ying Yee Wong, Alan Zhu, Lijie Yang, Xiaoxiang Shi, et al. Specinfer: Accelerating generative large language model serving with tree-based speculative inference and verification. *arXiv preprint arXiv:2305.09781*, 2023.

Xupeng Miao, Gabriele Oliaro, Zhihao Zhang, Xinhao Cheng, Hongyi Jin, Tianqi Chen, and Zhihao Jia. Towards efficient generative large language model serving: A survey from algorithms to systems. *ACM Computing Surveys*, 58(1):1–37, 2025.

Shen Nie, Fengqi Zhu, Zebin You, Xiaolu Zhang, Jingyang Ou, Jun Hu, Jun Zhou, Yankai Lin, Ji-Rong Wen, and Chongxuan Li. Large language diffusion models. *arXiv preprint arXiv:2502.09992*, 2025.

Jameson Sandler, Jacob K Christopher, Thomas Hartvigsen, and Ferdinando Fioretto. Specdiff-2: Scaling diffusion drafter alignment for faster speculative decoding. *arXiv preprint arXiv:2511.00606*, 2025.

Aaditya Singh, Adam Fry, Adam Perelman, Adam Tart, Adi Ganesh, Ahmed El-Kishky, Aidan McLaughlin, Aiden Low, AJ Ostrow, Akhila Ananthram, et al. Openai gpt-5 system card. *arXiv preprint arXiv:2601.03267*, 2025.

Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B. Hashimoto. Alpaca: A strong, replicable instruction-following model. *Stanford Center for Research on Foundation Models*, 2023. https://crfm.stanford.edu/2023/03/13/alpaca.html.

Tianyu Wu, Yu Yao, Zhenting Qi, Han Zheng, Zhuohan Wang, Haoran Ma, Lawrence Liao, Himabindu Lakkaraju, Ju Li, and Yilun Du. D-pace: Dynamic position-aware cross-entropy for parallel speculative drafting. *arXiv preprint arXiv:2605.18810*, 2026.

Tianjian Yang and Meng Li. Spec-auf: Accept-until-fail training under train-inference misalignment for masked block drafters. *arXiv preprint arXiv:2607.01893*, 2026.

Jiebin Zhang, Zhenghan Yu, Song Liu, Eugene J Yu, Zheng Li, Dawei Zhu, Jiangshan Duo, Weimin Xiong, Yifan Song, Guanghua Yu, et al. Dflare: Scaling up draft capacity for block diffusion speculative decoding. *arXiv preprint arXiv:2606.02091*, 2026a.

Yaojie Zhang, Jianuo Huang, Junlong Ke, Yuhang Han, Yongji Long, Tianchen Zhao, Biqing Qi, and Linfeng Zhang. Flexdraft: Flexible speculative decoding via attention tuning and bonus-guided calibration. *arXiv preprint arXiv:2605.20022*, 2026b.

Yilong Zhao, Jiaming Tang, Kan Zhu, Zihao Ye, Chi-Chih Chang, Chaofan Lin, Jongseok Park, Guangxuan Xiao, Mohamed S Abdelfattah, Mingyu Gao, et al. Accelerating large-scale reasoning model inference with sparse self-speculative decoding. *Proceedings of Machine Learning and Systems*, 8:251–265, 2026.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging llm-as-a-judge with mt-bench and chatbot arena. In *Advances in Neural Information Processing Systems*, 2023.

# A APPENDIX

## A.1 IMPLEMENTATION DETAILS

For EAGLE-3, we directly use the released model checkpoint provided by AngelSlim[2]. DFlash and DFlow are both trained on ShareGPT for 6 epochs with a learning rate of $6 \times 10^{-4}$. Unless otherwise specified, DFlash and DFlow use identical training data and optimization settings for a fair comparison. For evaluation, we set the maximum number of newly generated tokens to 2048 for all methods.

[2] https://github.com/Tencent/AngelSlim